

Design patterns & principes SOLID — Zappy

Document de synthèse pour la soutenance : quels **patrons de conception** et quels **principes SOLID** sont utilisés dans le projet, **expliqués simplement** et surtout **où** ils se trouvent (fichiers + extraits).

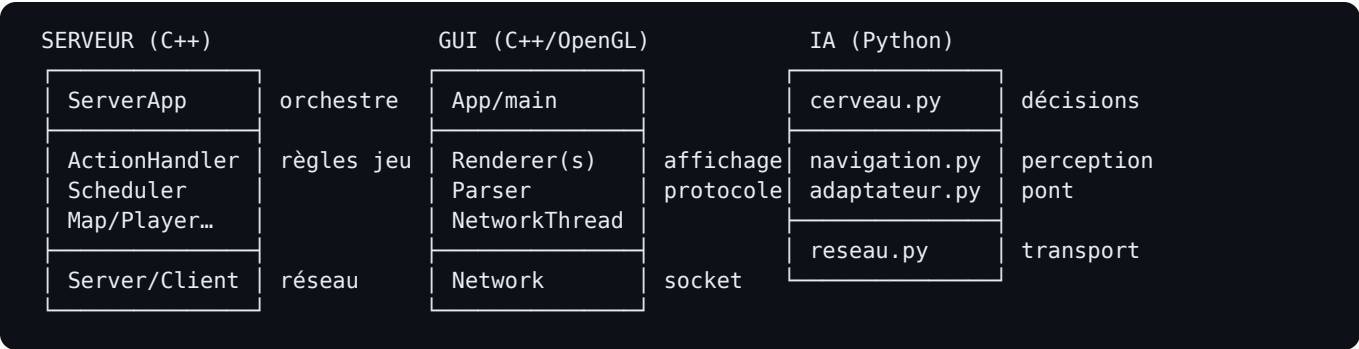
Le projet est découpé en **3 binaires** : `zappy_server` (C++), `zappy_gui` (C++/OpenGL) et `zappy_ai` (Python). Chacun illustre des patrons différents.

Sommaire

- A. Architecture en couches
- B. Les design patterns
 - B1. Abstraction par interfaces (Strategy)
 - B2. Observer / Callback (évènementiel)
 - B3. Command + Scheduler (file d'actions)
 - B4. Mediator / Facade (orchestration)
 - B5. Adapter (IA)
 - B6. State machine (IA)
 - B7. Producer-Consumer + Monitor (threads GUI)
 - B8. Dispatch table / Registry (parseur GUI)
 - B9. Facade de protocole
 - B10. RAI & Dependency Injection
- C. Les principes SOLID
- D. Autres principes
- E. Tableau récapitulatif

A. Architecture en couches

Chaque binaire est organisé en couches à responsabilité unique :



Idée : **séparer “qui parle au réseau”, “qui applique les règles” et “qui affiche/décide”**. C'est la base qui rend tous les patrons ci-dessous possibles.

B. Les design patterns

B1. Abstraction par interfaces

En une phrase : on programme contre des **interfaces** (classes abstraites pures), pas contre des classes concrètes — on peut remplacer l'implémentation sans toucher au reste.

Où : `zappy_server/include/interfaces/` → `IServer`, `IMap`, `IPlayer`, `IGameEngine`. Les classes concrètes en héritent : `Server : IServer`, `Map : IMap`, `Player : IPlayer`, `ServerApp : IGameEngine`.

```
// IMap : un contrat, pas une implémentation
class IMap {
public:
    virtual ~IMap() = default;
    virtual int getWidth() const = 0;
    virtual Tile &getTile(int x, int y) = 0;
    virtual Position wrap(const Position &pos) const = 0;
    // ...
};
```

```
class Map : public IMap { /* implémentation concrète */ };
```

C'est la base du **D** de SOLID (voir §C) : les modules de haut niveau manipulent `IMap`, jamais `Map` directement.

B2. Observer / Callback

En une phrase : un objet **notifie** des évènements à des fonctions abonnées, sans connaître qui les traite (découplage émetteur ↔ récepteur).

Où : - `Server` expose des points d'abonnement (`std::function`) :

```
// zappy_server/include/network/Server.hpp
void setOnConnect(std::function<void(Client &)> cb) { _onConnect = cb; }
void setOnDisconnect(std::function<void(Client &)> cb) { _onDisconnect = cb; }
void setOnData(std::function<void(Client &)> cb) { _onData = cb; }
void setOnStdinEOF(std::function<void()> cb) { _onStdinEOF = cb; }
```

- `ServerApp` s'y abonne sans que `Server` ne connaisse `ServerApp` :

```
// zappy_server/src/ServerApp.cpp
_server.setOnConnect ([this](Client &c){ onConnect(c); });
_server.setOnData ([this](Client &c){ onData(c); });
_scheduler->setOnActionReady([this](Player *p, ActionType t, auto &a){ onActionReady(p,t,a); });
_resourceManager->setOnSpawn([this](int x,int y){ /* notifie le GUI */ });
```

- `Session` fait pareil (`setOnTeamName`, `setOnGUI`, `setOnAICommand`, ...).

Bénéfice : le réseau ne dépend pas de la logique de jeu ; on pourrait brancher un autre « moteur » sur le même `Server`.

B3. Command + Scheduler

En une phrase : une action du joueur est **transformée en objet** (données) mis dans une file, puis **exécutée plus tard** (au bon moment selon la fréquence du serveur).

Où : `CommandScheduler` + `ScheduledAction` + `ActionHandler::execute` .

```
// L'action est un OBJET : qui, quoi, arguments, quand
struct ScheduledAction {
    Player *player; ActionType type;
    std::vector<std::string> args; double remainingTime; int id;
};
```

```
// 1) Une ligne réseau ("Forward") devient une commande planifiée
ParsedAction action = AIProtocol::parse(line);          // texte -> {type, args}
_scheduler->schedule(player, action.type, action.args); // mise en file (délai 7/f, etc.)

// 2) Plus tard, quand le délai est écoulé, on l'exécute
void ActionHandler::execute(Player &p, ActionType type, const std::vector<std::string>&args){
    switch (type) {
        case ActionType::FORWARD: handleForward(p); break;
        case ActionType::TAKE:    handleTake(p, args[0]); break;
        // ...
    }
}
```

C'est le patron **Command** (encapsuler une requête en objet) couplé à une **file de priorité** par temps d'exécution. Cela respecte aussi la règle Zappy : les actions ont un coût en temps et sont **séquentielles** par joueur.

B4. Mediator / Facade

En une phrase : un objet central **coordonne** plusieurs sous-systèmes qui, sinon, devraient se connaître les uns les autres.

Où : - `ServerApp` = **Facade** : il assemble et pilote `Server` , `Map` , `ResourceManager` , `CommandScheduler` , `ActionHandler` (composition) :

```
// zappy_server/include/ServerApp.hpp
Server _server;
std::unique_ptr<IMap> _map;          // <-- dépend de l'INTERFACE
std::unique_ptr<ResourceManager> _resourceManager;
std::unique_ptr<CommandScheduler> _scheduler;
std::unique_ptr<ActionHandler> _actions;
```

- `ActionHandler` = **Mediator** des règles : il met en relation map, équipes, joueurs, sessions, scheduler et incantations, pour exécuter une action sans que ces sous-systèmes se référencent mutuellement.

Bénéfice : un seul endroit connaît le « tout », les briques restent indépendantes et testables.

B5. Adapter

En une phrase : on **traduit** une interface existante vers l'interface qu'attend un autre code, sans modifier ni l'un ni l'autre.

Où : `zappy_ai/adaptateur.py` → `AdaptateurCerveau` . Le cerveau a été écrit pour une interface « mock » (`send` , `receive_response` , `connect`), alors que le vrai réseau (`reseau.py`) a une API différente (`envoyer_commande` , `recevoir_reponse` , file de 10 commandes...). L'adaptateur fait le pont :

```
class AdaptateurCerveau:
    def __init__(self, reseau):          # enveloppe l'objet réel
        self.reseau = reseau
    def send(self, commande):           # interface attendue par le cerveau
        morceaux = commande.strip().split()
        self.reseau.envoyer_commande(morceaux[0], *morceaux[1:]) # API réelle
    def receive_response(self, is_look=False, is_inventory=False):
        # ... traduit la réponse brute vers ce que le cerveau attend
```

C'est l'**Adapter** par excellence : `Brain` ne sait pas qu'il y a un vrai socket derrière.

B6. State machine

En une phrase : le comportement de l'IA dépend d'un **état courant** ; selon l'état, elle agit différemment, et des règles décident des transitions.

Où : `zappy_ai/cerveau.py` → `enum State` + `arbitrate()` (transitions) + `act()` (action selon l'état).

```
class State(Enum):
    SURVIVE = "survive"; COLLECT = "collect"; ELEVATE = "elevate"
    RALLY = "rally"; JOIN = "join"; REPRODUCE = "reproduce"; EXPLORE = "explore"

    def arbitrate(self):                # choisit l'état selon le contexte
        if food < self.HUNGER_CRITICAL: self.state = State.SURVIVE; return
        if self.player.can_elevate():   self.state = State.RALLY if req>1 else State.ELEVATE
        # ...

    def act(self):                      # agit selon l'état
        if self.state == State.SURVIVE: self.survive()
        elif self.state == State.RALLY: self.rally()
        elif self.state == State.JOIN:  self.do_join()
        # ...
```

C'est une **machine à états finis** : claire, extensible (ajouter un état = ajouter un cas), et facile à expliquer.

B7. Producer-Consumer + Monitor

En une phrase : un thread **produit** des données, un autre les **consomme**, et un objet **protégé par un verrou** (Monitor) assure que les deux ne se marchent pas dessus.

Où (GUI) : - Producteur : `NetworkThread` (lit le socket, remplit `GameState`). - Consommateur : `RenderThread` (lit l'état, dessine à 60 fps). - Monitor : `SharedState` et `CommandQueue` , chacun **protégé par un mutex**.

```
// zappy_gui/src/core/SharedState.hpp (objet Monitor)
void write(const GameState &s){ std::lock_guard<std::mutex> l(_mutex); _state = s; }
GameState read() { std::lock_guard<std::mutex> l(_mutex); return _state; }
```

```
// zappy_gui/src/network/CommandQueue.hpp (file thread-safe)
void push(const std::string &c){ std::lock_guard<std::mutex> l(_mtx); _queue.push(c); }
bool pop(std::string &out) { std::lock_guard<std::mutex> l(_mtx); /* ... */ }
```

Bénéfice : l'affichage ne bloque jamais sur le réseau, et l'état partagé reste cohérent.

B8. Dispatch table / Registry

En une phrase : au lieu d'un gros `if/else`, on associe chaque **mot-clé** à une **fonction** dans une table ; recevoir une commande = chercher + appeler.

Où : `zappy_gui/src/parser/Parser.cpp` + `CommandHandlers`.

```
// Enregistrement : commande -> fonction
_handlers[Protocol::PNW] = CommandHandlers::handlePnw;
_handlers[Protocol::PPO] = CommandHandlers::handlePpo;
_handlers[Protocol::PDI] = CommandHandlers::handlePdi;
// ...

// Dispatch : on lit le mot-clé et on appelle le bon handler
auto it = _handlers.find(cmd);
if (it != _handlers.end())
    it->second(ss, _state);
```

Bénéfice (= OCP) : ajouter une commande serveur = ajouter une ligne + une fonction, **sans modifier** la logique de dispatch.

B9. Facade de protocole

En une phrase : toute la connaissance du « format des messages » est regroupée dans des classes dédiées, le reste du code n'écrit jamais de texte brut.

Où : `AIProtocol` et `GUIProtocol` (serveur). Méthodes statiques `parse(...)` (texte → données) et `format...(())` (données → texte) :

```
// zappy_server/include/protocol/AIProtocol.hpp
static ParsedAction parse(const std::string &line); // "Forward" -> {FORWARD, {}}
static std::string formatOk(); // -> "ok\n"
static std::string formatBroadcast(int dir, const std::string &msg);
```

Bénéfice : si le protocole change, on ne touche qu'un fichier. Le reste manipule des `ParsedAction` / chaînes prêtes à l'emploi.

B10. RAII & Dependency Injection

RAII (idiome C++ central) : la **possession** d'une ressource est liée à la **durée de vie** d'un objet ; le destructeur libère automatiquement.

```
std::unique_ptr<IMap> _map;           // libéré tout seul
Server::~Server() { stop(); }        // ferme sockets + nettoie
Client::~Client() { disconnect(); }  // ferme le fd
~IslandRenderer() { _alien.destroy(); _egg.destroy(); /* libère le GPU */ }
```

Dependency Injection : un objet **reçoit** ses dépendances de l'extérieur (constructeur) au lieu de les créer lui-même :

```
// ActionHandler reçoit TOUT par référence (injection) -> testable, découplé
ActionHandler(IMap &map, std::vector<Team> &teams,
              std::vector<std::unique_ptr<Player>> &players, ...);
```

```
# IA : le cerveau reçoit son "réseau" et son "joueur" -> on peut injecter un mock
Brain(network, player, spawn_callback=spawn).run()
```

C. Les principes SOLID

S — Single Responsibility (responsabilité unique)

Chaque classe a **une seule raison de changer**.

Classe	Sa seule responsabilité
Client	l'I/O d'une connexion (lire/écrire le socket)
Server	accepter/ poll les connexions
Session	l'état protocolaire d'un client (handshake AI/GUI)
CommandScheduler	le timing des actions
ActionHandler	appliquer les règles des actions
IncantationSystem / LookSystem / BroadcastSystem	une mécanique de jeu chacun
ResourceManager	le spawn des ressources
Parser / CommandHandlers (GUI)	traduire le protocole en GameState
chaque *Renderer (GUI)	dessiner un type d'objet
reseau / navigation / cerveau (IA)	transport / perception / décision

O — Open/Closed (ouvert à l'extension, fermé à la modification)

On **ajoute** sans **modifier** l'existant : - Nouvelle commande GUI → ajouter une entrée dans la table `_handlers` (`Parser`), sans toucher `parse()` (cf. B8). - Nouvelle action serveur → ajouter une valeur à `ActionType` + un `case` dans `ActionHandler::execute` . - Nouveau rendu → ajouter une classe `*Render` sans modifier les autres. - Nouvel état IA → ajouter une valeur à `State` + un cas dans `act()` .

L — Liskov Substitution (substituabilité)

Partout on manipule la **classe de base** et n'importe quelle implémentation fonctionne : - `ServerApp` stocke `std::unique_ptr<IMap>` et n'utilise que l'interface. - `ActionHandler` reçoit `IMap &` : on pourrait passer une autre carte (ex. carte de test) sans rien casser. - `Server : IServer` , `Player : IPlayer` , `ServerApp : IGameEngine` sont utilisables via leur interface.

I — Interface Segregation (interfaces fines)

Les interfaces sont **petites et ciblées**, pas un gros « god object » : - `IServer` = 4 méthodes (init/run/stop/isRunning). - `IMap` , `IPlayer` , `IGameEngine` séparées : un client réseau n'a pas à connaître les règles du jeu, et inversement.

D — Dependency Inversion (dépendre des abstractions)

Le **haut niveau** dépend d'**abstractions**, pas de détails : - `ServerApp` / `ActionHandler` dépendent de `IMap` , pas de `Map` . - `Server` dépend de `std::function` (le « quoi faire ») et non de `ServerApp` : c'est `ServerApp` qui s'injecte via `setOnConnect/...` (cf. B2). - L'IA `Brain` dépend d'un objet « réseau » abstrait (l' `AdaptateurCerveau`), injecté au constructeur → testable avec un mock (cf. B5/B10).

```
Haut niveau (règles, décision)
  | dépend de ▼ (interface)
IMap / std::function / "network"
  ▲ implémenté par
Bas niveau (Map, ServerApp, Réseau)
```

D. Autres principes

- **Separation of Concerns** : 3 binaires + couches internes (réseau / logique / affichage-décision) ne se mélangent jamais.
- **Single Source of Truth** : l'état du jeu vit à un seul endroit (serveur = vérité ; côté GUI, `GameState.players/tiles/eggs` ; la vie = `Inventory[F00D]`).
- **DRY** (Don't Repeat Yourself) : le format réseau est centralisé dans `AIProtocol` / `GUIProtocol` ; les coordonnées case → monde via une seule formule (`offset + i * ISLAND_SPACING`) ; `extractId` factorise le retrait du `#` .
- **Fail-safe / robustesse** : `try/catch` autour des handlers GUI, gestion `EAGAIN` , garde-fous mémoire sur les buffers — le programme ne crashe pas sur une entrée inattendue.

E. Tableau récapitulatif

Pattern / Principe	Où (fichiers clés)	Rôle
Interfaces / Abstraction	<code>interfaces/I*.hpp</code>	contrats remplaçables
Observer / Callback	<code>Server.hpp</code> , <code>Session</code> , <code>ServerApp.cpp</code>	événements découplés
Command + Scheduler	<code>CommandScheduler</code> , <code>ActionHandler</code>	actions différées, séquentielles
Mediator / Facade	<code>ServerApp</code> , <code>ActionHandler</code>	orchestration
Adapter	<code>zappy_ai/adaptateur.py</code>	pont cerveau ↔ réseau réel
State machine	<code>zappy_ai/cerveau.py</code> (<code>State</code>)	comportement IA
Producer-Consumer	<code>NetworkThread</code> / <code>RenderThread</code>	réseau vs rendu
Monitor object	<code>SharedState</code> , <code>CommandQueue</code>	accès concurrent sûr
Dispatch table	<code>Parser.cpp</code> (<code>_handlers</code>)	protocole GUI extensible
Facade de protocole	<code>AIProtocol</code> , <code>GUIProtocol</code>	format réseau centralisé
RAII	<code>unique_ptr</code> , destructeurs	gestion auto des ressources
Dependency Injection	<code>ActionHandler(...)</code> , <code>Brain(...)</code>	découplage, testabilité
SOLID – S	toutes les classes ci-dessus	1 responsabilité chacune
SOLID – O	<code>Parser</code> , <code>ActionType</code> , <code>*Renderer</code> , <code>State</code>	extension sans modif
SOLID – L	<code>IMap</code> / <code>IServer</code> / <code>IPlayer</code>	implémentations substituables
SOLID – I	<code>interfaces/I*.hpp</code>	interfaces fines
SOLID – D	<code>IMap</code> dans <code>ServerApp</code> , callbacks <code>Server</code> , IA <code>Brain</code>	dépendre des abstractions

Phrase de synthèse pour le jury : « Le projet repose sur une architecture en couches découplées ; les **interfaces** + l'**injection de dépendances** donnent le D et le L de SOLID, l'**Observer** relie réseau et logique sans couplage, le **Command/Scheduler** modélise les actions temporisées du serveur, l'**Adapter** et la **machine à états** structurent l'IA, et le couple **Producer-Consumer / Monitor** rend le GUI fluide et thread-safe. »